

Term Project Final Report

Title: Predicting Late Delivery Risk in the Supply Chain Dataset

Team Member: Shreyas Basavaraju Ravindra

Table of Contents

1. Executive Summary
2. Introduction & Motivation
3. Objective
4. Dataset Overview & Rationale
5. Data Preprocessing & Cleaning
 - Feature Selection
 - Initial Sanity Checks
 - Exploratory Data Analysis
 - Feature Engineering
 - Encoding, Scaling, and Class Balancing
6. Model Building and Evaluation
 - Logistic Regression
 - Random Forest Classifier
 - XGBoost Classifier
7. Performance Comparison
8. Reflection & Future Work
9. Conclusion
10. References
11. Appendix

1. Executive Summary

This project investigates the feasibility of using machine learning to predict late delivery risk in a global supply chain operation. Leveraging the DataCo Smart Supply Chain dataset, three classification models were developed and rigorously compared: Logistic Regression, Random Forest, and XGBoost. The primary objective was to accurately classify whether an order would be delivered late ($\text{Late_delivery_risk} = 1$) or on time ($\text{Late_delivery_risk} = 0$) based solely on information available at the moment the order is placed.

The analysis followed a comprehensive pipeline that included careful feature selection designed to avoid data leakage, removal of 30,561 duplicate rows, extensive exploratory data analysis, feature engineering, log transformation of skewed variables, one-hot encoding of categorical features, standardization of numerical variables, Hyperparameter tuning was performed for both ensemble models using cross-validated grid and randomized search strategies.

After thorough preparation and evaluation, the Random Forest classifier emerged as the strongest performer, achieving a test set accuracy of **68.55%**, precision of **82.72%**, recall of **56.26%**, and F1-score of **66.97%**. All three models converged to closely comparable test performance in the 68–69% accuracy range, but **Random Forest** delivered the marginal lead on accuracy and recall, while Logistic Regression achieved the highest precision (83.88%) and XGBoost the highest F1-score by a hair (66.99%). Feature importance analysis identified shipping mode (particularly First Class and Second Class), scheduled shipping days, and order region as the dominant predictors of delivery delay. These findings demonstrate the practical value of predictive analytics for supply chain operations and identify clear ways on how a business can use these findings to increase its margins.

2. Introduction & Motivation

Late deliveries are a persistent and costly challenge in modern supply chain management. They erode customer trust, trigger contractual penalties, inflate logistics costs through expedited corrective shipping, and disrupt downstream operations for both retailers and end customers. As global supply chains have grown more complex, with goods flowing across multiple markets, regions, and carriers the ability to anticipate delays before they occur has become a meaningful competitive advantage.

Although modern enterprises collect enormous volumes of operational data covering shipments, products, customers, and sales, that data is often used reactively for post-event reporting rather than proactively for prediction. Machine learning offers a path to change this. By learning historical patterns in operational variables like payment type, shipping mode, scheduled lead time, product category, geographic region, customer segment, and order economics predictive models can flag at-risk orders at the moment of fulfillment, before delays materialize.

The DataCo Smart Supply Chain dataset provides a realistic, full-spectrum view of these operations. It captures shipping performance, product characteristics, customer information, financial outcomes, and delivery results for a global retail operation. This makes it a strong candidate for studying late delivery risk in a way that mirrors real business conditions. This project examines whether classification models trained on order-time information alone can reliably distinguish late from on-time orders, and which operational factors contribute most strongly to delay risk.

3. Objective

This project was designed to answer four central questions that together establish the feasibility, mechanisms, and operational value of predicting late-delivery analytics:

1. Can late delivery risk be predicted accurately using order, shipment, product, and customer-related variables?

The first question tests whether the information available at the moment an order is placed contains enough signal to discriminate late from on-time orders without relying on post-shipment fields that would constitute data leakage. A definitive answer requires building and honestly evaluating classification models on a held-out test set.

2. Which factors contribute most to late delivery risk, such as shipping mode, scheduled shipping days, market, order region, customer segment, or product category?

The second question seeks to identify the dominant operational drivers of delay. This is investigated both through bivariate exploratory data analysis (comparing late-delivery rates across each categorical feature) and through the feature importance scores produced by the trained Random Forest model.

3. Do some shipping modes, markets, or order regions experience higher levels of late delivery risk than others?

The third question investigates whether observed delay rates differ systematically across logistical and geographic strata. Where such differences exist, they indicate where corrective operational effort adjusted lead-time promises, carrier renegotiation, regional capacity reinforcement should be concentrated.

4. Which machine learning model provides the best performance for predicting late delivery risk in this dataset?

The fourth question evaluates three distinct modeling approaches Logistic Regression (interpretable linear baseline), Random Forest (robust ensemble), and XGBoost (gradient boosting) on test set accuracy, precision, recall, and F1-score to identify the strongest performer and determine whether nonlinear modeling capacity is necessary for this prediction task.

4. Dataset Overview & Rationale

The primary dataset for this project is the DataCo Smart Supply Chain dataset, sourced from Kaggle. The full dataset contains 180,519 records and 53 columns, spanning customer orders, products, shipping, delivery outcomes, and financial metrics for a global retail operation across multiple markets and regions.

The dataset was selected for three reasons. First, the late delivery problem it captures is a high-value, business-relevant prediction task with direct implications for shipment planning, customer satisfaction, and operational efficiency. Second, the dataset contains a rich mix of categorical variables (payment type, shipping mode, product category, customer segment, geography) and numerical variables (scheduled lead time, order quantity, discount rate, product price), making it well suited for comparing classification models that handle different feature types in different ways. Third, the target variable `Late_delivery_risk` is reasonably balanced which permits meaningful evaluation across accuracy, precision, recall, and F1 without severe imbalance distortions.

From the original 53 columns, **12 features** were retained for modeling. Feature selection was driven by two principles: relevance to delivery outcomes, and strict avoidance of data leakage. Variables such as **Days for shipping (real)** and **Delivery Status** were deliberately excluded because they are derived after shipment completion and would directly reveal the target. Including them would produce artificially perfect accuracy and an operationally meaningless model. The final 12 features all reflect information knowable at the moment an order is placed.

Selected Features

#	Feature	Description
1	Type	Payment / transaction type (DEBIT, CASH, TRANSFER, PAYMENT)
2	Days for shipment (scheduled)	Number of days the order was scheduled to take
3	Shipping Mode	Mode of shipment (Standard, First, Second, Same Day)
4	Category Name	Product category (top 10 retained, remaining grouped as 'Other')
5	Department Name	Product department
6	Customer Segment	Segment of the customer (Consumer, Corporate, Home Office)
7	Customer Country	Country of the customer
8	Market	Market region (Africa, Europe, LATAM, Pacific Asia, USCA)
9	Order Region	Geographic region of the order (23 distinct regions)
10	Order Item Quantity	Quantity of items ordered
11	Order Item Discount Rate	Discount rate applied to the order item

#	Feature	Description
12	Product Price	Listed price of the product (log-transformed during preprocessing)

The target variable, `Late_delivery_risk`, is a binary indicator where 1 denotes a late delivery and 0 denotes an on-time delivery.

5. Data Preprocessing & Cleaning

I built a preprocessing pipeline to turn the raw data into a clean, leak-free, and model ready dataset for classification. I did everything in one Python notebook using pandas, NumPy, scikit-learn, imbalanced-learn, and XGBoost. The process had five main steps: feature selection, sanity checks, exploratory analysis, feature engineering, and encoding plus scaling.

Feature Selection

The dataset was first reduced from 53 columns to the 12 features listed in Section 4, plus the target variable. Variables that record post-shipment outcomes (Days for shipping (real), Delivery Status, shipping date) were dropped to prevent data leakage. High-cardinality identifier fields (Customer Id, Order Id, Customer Email, Customer Password, Product Name, Order City) were also excluded because they offer little generalizable signal and would only inflate the feature space after one-hot encoding.

Initial Sanity Checks

After feature selection, the dataset was inspected for duplicates and missing values. A check using pandas' `duplicated().sum()` method identified 30,561 fully duplicate rows, which were subsequently removed using `drop_duplicates()`, reducing the working dataset from 180,519 records to 149,958 records. A null check across all retained columns confirmed that no missing values remained, eliminating the need for imputation. The data types of all columns were also verified to confirm that numerical features were correctly typed as numeric and categorical features as object, supporting the encoding steps that followed.

Exploratory Data Analysis

A thorough exploratory data analysis was conducted to examine the distributions of the variables and their relationships with the target outcome. For the numerical features, descriptive statistics, histograms, and boxplots were used to assess the data. Product Price was notably right-skewed, while Days for shipment (scheduled) ranged from 0 to 4 days and Order Item Quantity and Order Item Discount Rate showed relatively limited variation. The categorical variables were then analyzed using count plots and stacked bar charts against the binary delivery outcome, revealing clear differences across groups. Among these, Shipping Mode showed the strongest relationship with late delivery, with late-delivery rates varying by about 55 percentage points.

Table 1

Shipping Mode	Late Rate	On-Time Rate
First Class	94.88%	5.12%
Second Class	75.09%	24.91%
Same Day	45.92%	54.08%
Standard Class	40.02%	59.98%

This result is operationally important and somewhat counter-intuitive: faster shipping modes (First Class and Second Class) experienced dramatically higher late rates than slower modes (Standard Class). The most plausible explanation is that faster modes have tighter scheduled lead times and therefore far less slack to absorb operational variance when anything goes wrong upstream, a First Class delivery has no buffer.

By contrast, Market region showed only mild variation, with late rates ranging narrowly from 55.5% (Africa) to 57.2% (Europe). Order Region showed a wider spread, from 49.1% late (Canada the only region where on-time deliveries exceeded late ones) to 58.5% late (Central Africa) and 58.4% late (Western Europe). Payment Type showed a moderate signal: TRANSFER deliveries were late just 50.6% of the time, while DEBIT deliveries were late 59.5% of the time. Customer Segment, Customer Country, Category Name, and Department Name all showed only minor differences, suggesting that these features carry less predictive signal individually but may still contribute through interactions.

A pairplot showed that no single numerical feature clearly separated late and on-time orders, which suggests nonlinear models may work better. A correlation heatmap also showed little multicollinearity among the retained predictors.

Feature Engineering

Three feature engineering steps were applied. First, Category Name originally contained 50 distinct product categories, with a long tail of low-frequency entries. To control encoded dimensionality and reduce noise from rare categories, only the top 10 categories by frequency were retained, with all remaining categories collapsed into a single 'Other' bucket — leaving 11 levels in the final feature. Second, the Customer Country field contained the value 'EE. UU.' (a Spanish-language abbreviation for the United States) which was standardized to 'United States' to ensure consistent encoding. Third, Product Price exhibited strong right-skew with many high-priced outliers. To stabilize variance and reduce the influence of extreme values, a log transformation (np.log1p) was applied to produce Product Price (log), and the original Product Price variable was

dropped from the modeling feature set. The transformed distribution was substantially more symmetric, which is generally beneficial for linear models and does not harm tree-based models.

Encoding, Scaling, and Class Balancing

Categorical features were converted to numerical form using one-hot encoding with `drop_first = True` to prevent the dummy variable trap. The resulting feature matrix expanded from 12 base features to 59 columns and was cast to float for downstream compatibility. The dataset was then split into a training set (70%) and a test set (30%) using a stratified split on the target variable to preserve the class distribution in both splits. A random state of 42 was fixed throughout to ensure reproducibility.

Standardization was applied only to the four numerical features (Days for shipment (scheduled), Order Item Quantity, Order Item Discount Rate, and Product Price (log)) using scikit-learn's `StandardScaler`. The scaler was fit exclusively on the training set and then used to transform both training and test sets, ensuring that no information from the test set leaked into the scaling parameters. One-hot encoded dummy columns were left unscaled.

Finally, although the class distribution in the dataset was only mildly imbalanced (approximately 56.7% late vs. 43.3% on-time), *SMOTE* (*Synthetic Minority Over-sampling Technique*) was applied to the training data with `sampling_strategy = 1` and `k_neighbors = 5` to produce a perfectly balanced training set. Before oversampling, the training set contained 59,495 late and 45,475 on-time examples; after oversampling, both classes contained 59,495 examples, yielding a final training matrix of shape (118,990, 59). Importantly, SMOTE was applied **after** the train-test split and **only to the training set**, so the test set retained its original class distribution and remained a faithful proxy for real-world conditions.

6. Model Building and Evaluation

Three classification models were trained on the scaled, SMOTE-balanced training set and evaluated on the held-out test set. For each model, performance was measured using four standard classification metrics accuracy, precision, recall, and F1-score on both the training and test sets to detect overfitting. Confusion matrices were also generated to inspect the distribution of true positives, false positives, true negatives, and false negatives.

Hyperparameter Tuning Summary

Table 2:

Model	Tuning Strategy	Selected Hyperparameters
Logistic Regression	Default parameters (baseline)	solver = lbfgs, random_state = 42; no tuning
Random Forest	GridSearchCV (5-fold, accuracy scoring, 27 combinations $\times$ 5 folds = 135 fits)	n_estimators = 100, max_depth = 15, min_samples_split = 2
XGBoost	RandomizedSearchCV (5-fold, 30 iterations $\times$ 5 folds = 150 fits, accuracy scoring)	n_estimators = 300, learning_rate = 0.21, max_depth = 3, min_child_weight = 1, gamma = 0.2, colsample_bytree = 0.7

6.1 Logistic Regression

Logistic Regression was selected as the baseline classifier. It is a linear, fully interpretable model that estimates the probability of the positive class using a logistic function over a weighted combination of features. Its coefficients directly indicate the direction and magnitude of each feature's effect on the predicted log-odds of late delivery, which is operationally valuable for stakeholder communication. The model was trained on the scaled, oversampled training set using scikit-learn's default lbfgs solver and a fixed random state of 42

On the test set, Logistic Regression achieved an accuracy of 68.48%, precision of 83.88%, recall of 54.95%, and F1-score of 66.40%. Training set performance was closely aligned (accuracy 70.47%, F1 65.09%), indicating no meaningful overfitting. Notably, Logistic Regression achieved the highest precision of any model in the comparison when it predicts an order will be late, it is correct 83.88% of the time.

6.2 Random Forest Classifier

Random Forest was selected for its ability to capture nonlinear relationships and feature interactions, handle mixed feature types natively, and resist overfitting from outliers or irrelevant features. Hyperparameters were tuned via GridSearchCV.

Random Forest achieved test accuracy of 68.55%, precision of 82.72%, recall of 56.26%, and F1-score of 66.97% the strongest overall performance. Compared to Logistic Regression. The train-test gap (73.01% → 68.55%) indicates mild overfitting, well-controlled by cross-validation.

6.3 XGBoost Classifier

XGBoost was selected because gradient boosting has consistently achieved state-of-the-art performance on tabular tasks. Unlike Random Forest's independent trees, XGBoost trains trees sequentially to correct residual errors, using regularization to avoid overfitting. This approach often yields stronger performance but requires more hyperparameter tuning.

Due to the high-dimensional hyperparameter space, RandomizedSearchCV was used instead of exhaustive grid search.

XGBoost achieved test accuracy of 68.30%, precision of 81.75%, recall of 56.75%, and F1-score of 66.99% the highest recall and a marginally higher F1 than Random Forest (66.99% vs. 66.97%). The shallow optimal depth suggesting that the dataset's predictive signal comes from a few dominant features (shipping mode, scheduled days, region) rather than deep interactions.

7. Performance Comparison

Table 3 shows the training and test performance for of all three models on the four primary classification metrics; Random Forest is highlighted as the best-performing model overall.

Table 3: Test Set Performance Comparison

Model	Accuracy	Precision	Recall	F1 Score
Logistic Regression	68.48%	83.88%	54.95%	66.40%
Random Forest	68.55%	82.72%	56.26%	66.97%
XGBoost	68.30%	81.75%	56.75%	66.99%

Table 4: Training vs. Test Set Performance

Model (Split)	Accuracy	Precision	Recall	F1 Score
Logistic Regression- Train	70.47%	79.60%	55.05%	65.09%
Logistic Regression- Test	68.48%	83.88%	54.95%	66.40%
Random Forest- Train	73.01%	82.68%	58.21%	68.32%
Random Forest -Test	68.55%	82.72%	56.26%	66.97%
XGBoost - Train	72.48%	82.27%	57.31%	67.56%
XGBoost - Test	68.30%	81.75%	56.75%	66.99%

The three models converged to similar test set performance, with all four metrics tightly clustered: test accuracy ranged from 68.30% (XGBoost) to 68.55% (Random Forest), precision from 81.75% to 83.88%, recall from 54.95% to 56.75%, and F1-score from 66.40% to 66.99%. Random Forest delivered the marginal lead on accuracy and was the strongest performer overall, although XGBoost achieved a fractionally higher F1-score (a difference of 0.02 percentage points) and the highest recall, while Logistic Regression achieved the highest precision.

Comparing training and test performance reveals the degree of overfitting in each model. Logistic Regression showed a small train-test accuracy gap of about 2 percentage points, consistent with its limited model capacity. Random Forest had a slightly larger gap of approximately 4.5

percentage points, and XGBoost a similar gap of 4.2 percentage points both well within the acceptable range for ensemble methods and indicating that the cross-validated hyperparameter selection successfully prevented severe overfitting.

One pattern is shared across all three models: precision is consistently much higher than recall. This is operationally meaningful: when these models predict a late delivery, they are usually right, but they miss roughly 44% of all true late deliveries. Lowering the prediction threshold below 0.5 would shift this balance, sacrificing some precision to recover additional late deliveries. Section 8 returns to this point as a near-term opportunity for refinement.

Feature Importance Analysis

Feature importance was extracted from the best-performing Random Forest model. Consistent with the bivariate EDA in Section 5, the top-ranked features were dominated by shipping-related variables in particular, the dummy indicators for First Class and Second Class shipping modes and by scheduled shipping days. These features capture the operational reality that fast shipping modes have no slack to absorb upstream variance and therefore produce dramatically higher late rates. Order region dummies (notably Canada, Western Europe, and Central Africa) and payment type indicators (notably TRANSFER) contributed measurable secondary signal. Product-related variables (category and department dummies) contributed least, mirroring the small bivariate differences observed in EDA.

8. Reflection & Future Work

This project showed that late delivery can be predicted with reasonable accuracy using only order-time information. The ensemble models outperformed the linear baseline, confirming that shipping mode, geography, and scheduled days interact in nonlinear ways. By excluding post-shipment variables, the model stays operationally usable.

The biggest opportunity for improvement is **threshold tuning**. All three models showed high precision but moderate recall (~55%), meaning they catch only about half of actual late deliveries. The default 0.5 threshold isn't optimal here. Plotting precision-recall curves and selecting a threshold that maximizes F1 or hits a target recall (~75%) would make the model more useful in practice.

Class balancing may have been unnecessary. The original split was 57% late vs. 43% on-time were barely imbalanced. SMOTE forced a 50/50 balance, which might have hurt calibration more than it helped performance.

I tuned on **accuracy**, but minimizing false negatives (missed late deliveries) probably matters more than minimizing false positives. Tuning on F1 or a custom cost function would better match real operational priorities.

Temporal features were deliberately excluded to avoid leakage, but safe time features day of week, month, quarter, holiday flags could capture seasonality and capacity bottlenecks.

Alternative boosting frameworks might avoid the dimensionality explosion from one-hot encoding (12 features → 59 columns). And **lag features** (recent late rate by region or carrier) could capture correlated patterns that the current independent-order setup misses.

9. Conclusion

This project built a complete ML pipeline for predicting late deliveries using only order-time information no post-shipment data leakage. The approach was clean: feature selection, thorough EDA, proper train/test splitting, scaling, SMOTE balancing, and cross-validated tuning for both ensemble models.

Returning to the four questions:

1. Can late delivery be predicted accurately?

- Yes. The best model (Random Forest) hit 68.55% test accuracy using only information available when the order is placed.

2. Which factors matter most?

- Shipping mode dominates. First Class is late 95% of the time vs. 40% for Standard Class. Scheduled shipping days and order region also contribute significantly.

3. Do shipping modes, markets, and regions differ?

- Dramatically. Shipping mode spans 40–95%. Order regions range from 49% (Canada) to 58% (Central Africa). Markets are fairly flat (55–57%).

4. Which model wins?

- Random Forest, narrowly. It beat XGBoost on accuracy and recall, though XGBoost had slightly higher F1 and Logistic Regression had the highest precision.

The takeaway by using predictive analytics we can turn something as routine supply chain data into a useful early-warning system when you avoid leakage and engineering features carefully.

The next step threshold tuning, time features, and better categorical handling would make the analysis more put together.

10. References

- Constante, F., Silva, F., & Pereira, A. (2019). *DataCo Smart Supply Chain for Big Data Analysis* [Data set]. Mendeley Data. Available via Kaggle:
<https://www.kaggle.com/datasets/evilspirit05/datasupplychain>
- Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794.
<https://doi.org/10.1145/2939672.2939785>
- Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32.
<https://doi.org/10.1023/A:1010933404324>
- Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16, 321–357.
<https://doi.org/10.1613/jair.953>
- Brzozowska, J., Pizoń, J., Baytikenova, G., Gola, A., Zakimova, A., & Piotrowska, K. (2023). Data engineering in a smart supply chain management system: A case study of artificial neural networks for predicting late deliveries. *Advances in Science and Technology Research Journal*, 17(5), 184–196.

11. Appendix

The following figures supplement the analysis described in the body of the report. Each was generated as part of the project notebook and is referenced in the corresponding section.

Figure 1: Distributions of Numerical Variables

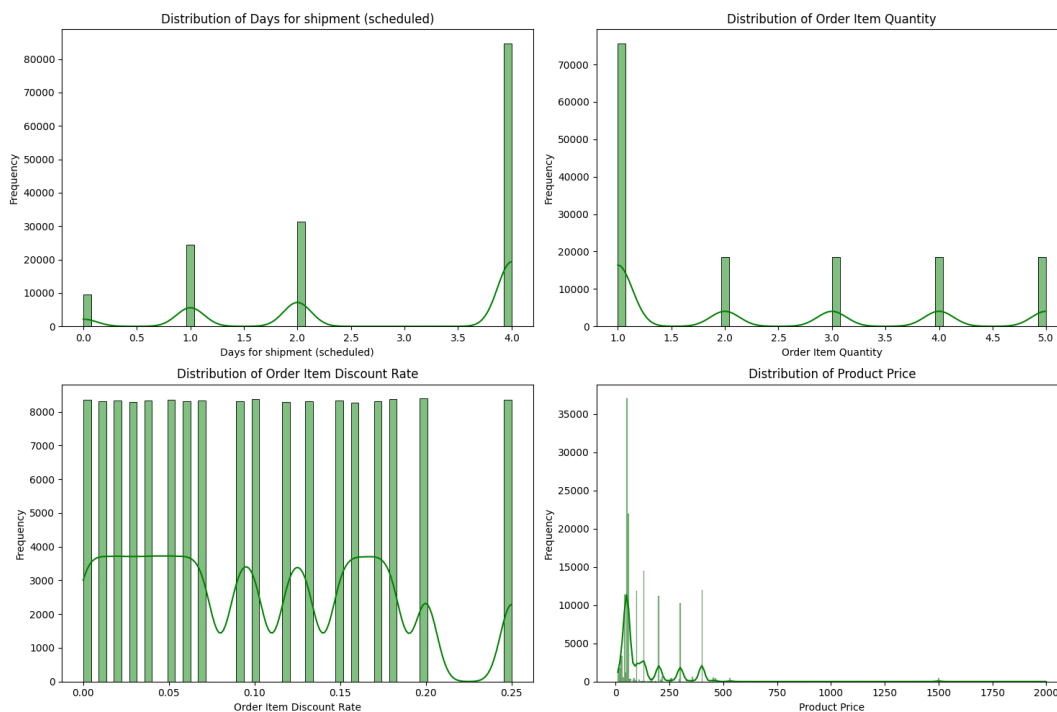
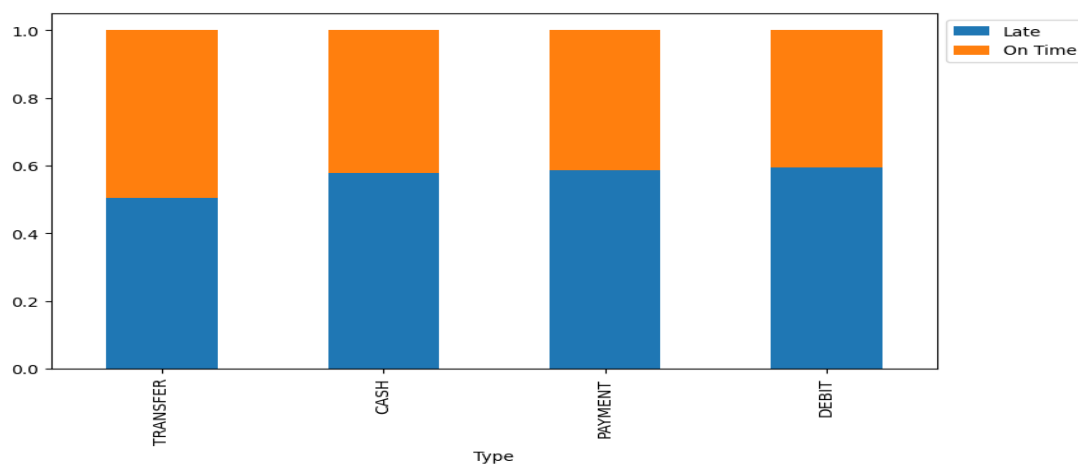


Figure 2: Stacked Bar Plots of Categorical Variables by Delivery Outcome



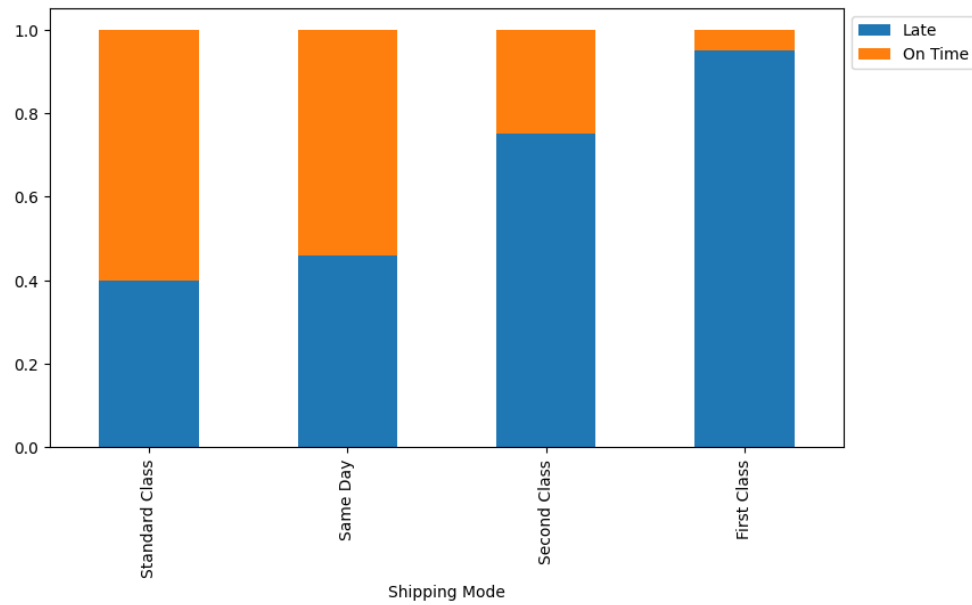


Figure 3: Correlation Matrix of Numerical Variables

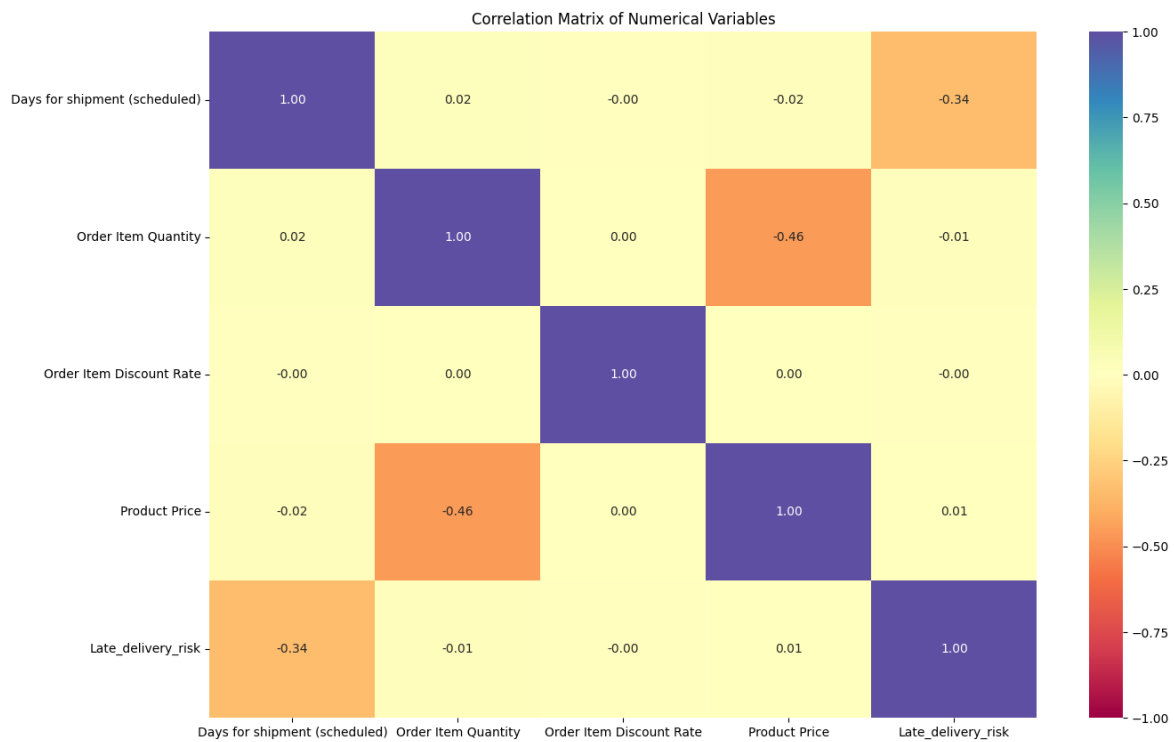


Figure 4: Confusion Matrices for Random Forest Model(Train)

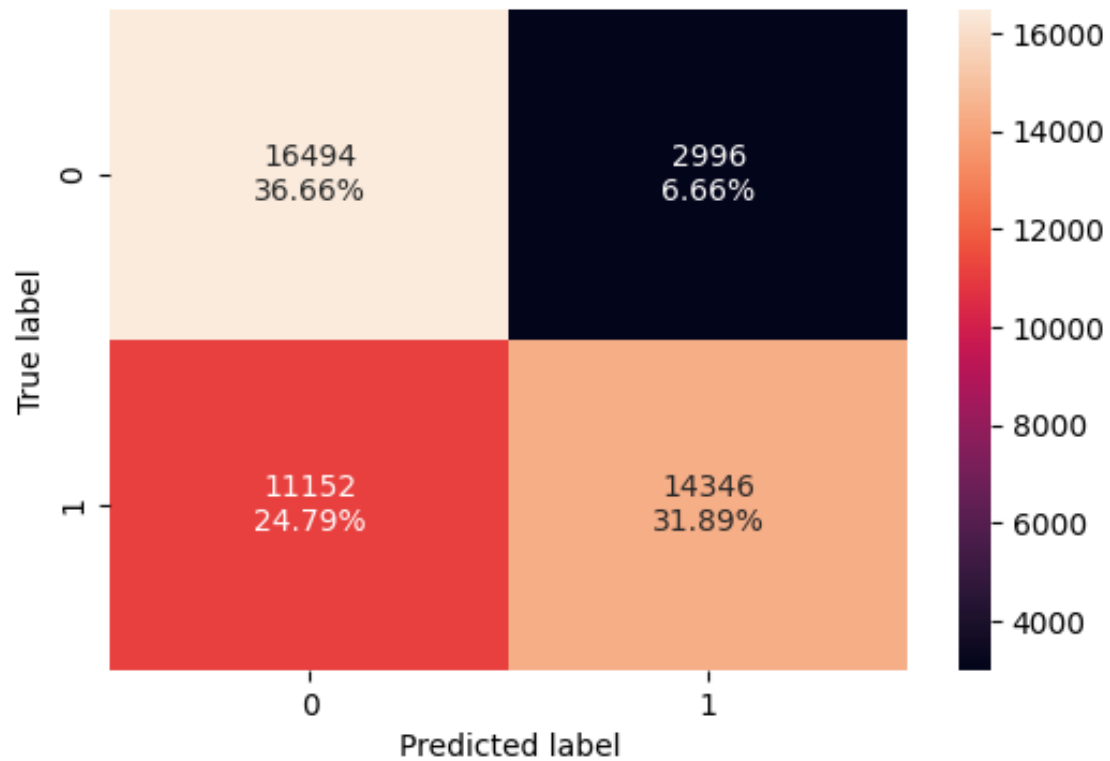


Figure 6: Feature Importance from Random Forest

